



SW Pilot 학습보고서 <과정 4>

파이썬 5팀 황문규

문제 1. 스페이스 타닉 사건 - 코드

```
1 # 과제 4 - (문제1) "스페이스 타이타닉 사건"
2
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import matplotlib
6
7 matplotlib.rc('font', family='AppleGothic')
8 plt.rcParams['axes.unicode_minus'] = False
9
10 def load_and_merge(train_path, test_path):
11     train_df = pd.read_csv(train_path)
12     test_df = pd.read_csv(test_path)
13     test_df['Transported'] = pd.NA # 테스트 데이터에 없는 열 추가 (Transported)
14     merged_df = pd.concat([train_df, test_df], ignore_index=True)
15     return merged_df
16
17
18 def report_shape(df):
19     print(f'총 행 개수: {df.shape[0]}')
20     print(f'총 열 개수: {df.shape[1]}')
21
22
23 def analyze_correlation(df):
24     df_numeric = df.copy()
25     df_numeric['Transported'] = df_numeric['Transported'].map({True: 1, False: 0})
26     correlation = df_numeric.corr(numeric_only=True)
27     correlation_target = correlation['Transported'].drop(labels=['Transported']).sort_values(ascending=False)
28
29     print('\n[Transported와의 상관계수 상위 5개]')
30     print(correlation_target.head(5))
31
32
33 def plot_age_groups(train_df):
34     bins = [0, 19, 29, 39, 49, 59, 69, 79]
35     labels = ['10대', '20대', '30대', '40대', '50대', '60대', '70대']
36     train_df['AgeGroup'] = pd.cut(train_df['Age'], bins=bins, labels=labels, right=True)
37     age_group_transport = train_df.groupby('AgeGroup')['Transported'].mean()
38
39     plt.figure(figsize=(10, 6))
40     age_group_transport.plot(kind='bar', color='skyblue', edgecolor='black')
41     plt.title('연령대별 Transported 비율')
42     plt.xlabel('연령대')
43     plt.ylabel('Transported 비율')
44     plt.ylim(0, 1)
45     plt.grid(axis='y')
46     plt.tight_layout()
47     plt.show()
```

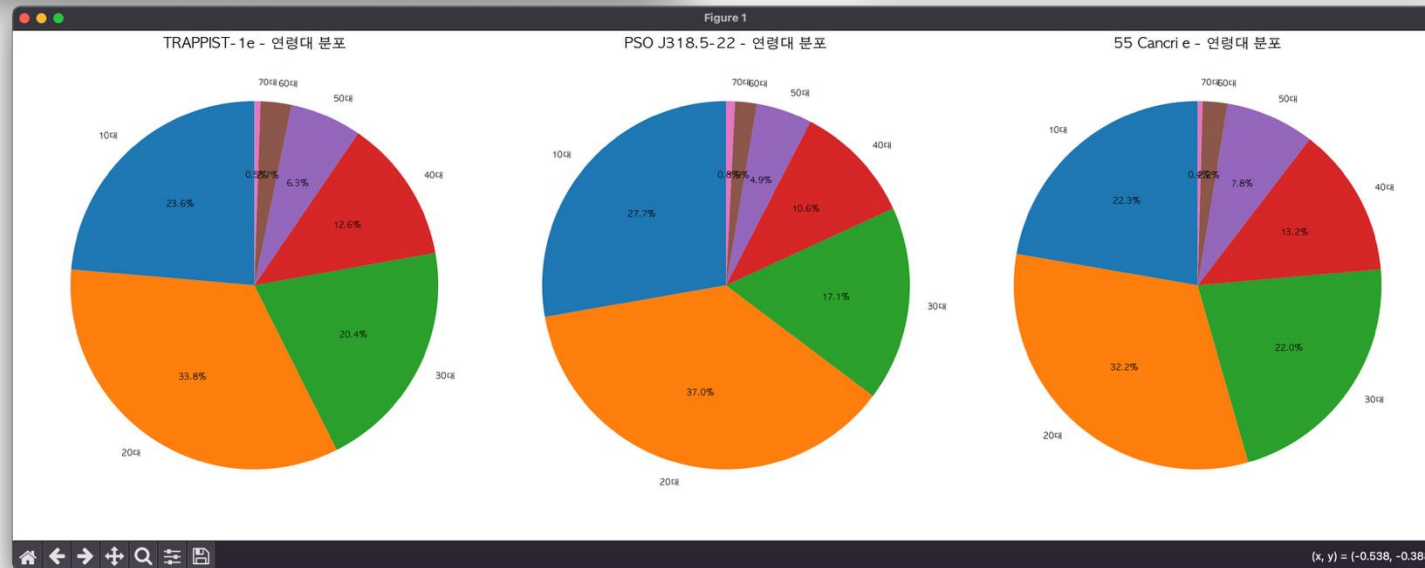
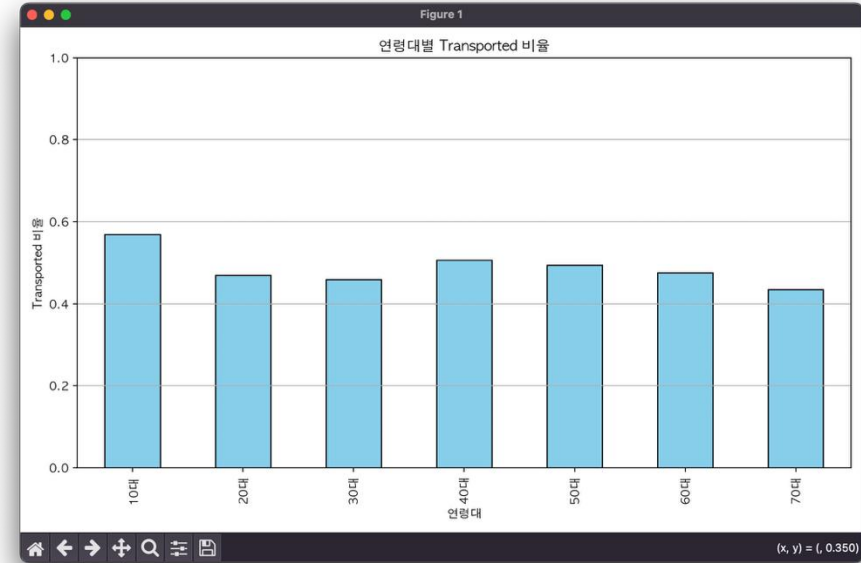
```
1 # [보너스 과제] - Destination 별로 승객들의 연령대 분포 시각화
2 def plot_destination_age_pie(df):
3     df = df[df['Age'].notna() & df['Destination'].notna()]
4
5     bins = [0, 19, 29, 39, 49, 59, 69, 79]
6     labels = ['10대', '20대', '30대', '40대', '50대', '60대', '70대']
7     df['AgeGroup'] = pd.cut(df['Age'], bins=bins, labels=labels, right=True)
8
9     # Destination별 연령대 분포 시각화 (원형 그래프)
10    destinations = df['Destination'].unique()
11    fig, axes = plt.subplots(1, len(destinations), figsize=(6 * len(destinations), 6))
12
13    if len(destinations) == 1:
14        axes = [axes]
15
16    for ax, dest in zip(axes, destinations):
17        group_counts = df[df['Destination'] == dest]['AgeGroup'].value_counts().sort_index()
18        ax.pie(
19            group_counts,
20            labels=group_counts.index,
21            autopct='%1.1f%%',
22            startangle=90,
23            textprops={'fontsize': 7},
24        )
25        ax.set_title(f'{dest} - 연령대 분포')
26
27    plt.tight_layout()
28    plt.show()
29
30 def main():
31     train_path = 'src/Process4/4-1/train.csv'
32     test_path = 'src/Process4/4-1/test.csv'
33
34     merged_df = load_and_merge(train_path, test_path)
35
36     print('데이터 수량')
37     report_shape(merged_df)
38
39     print('\n[상관 분석]')
40     analyze_correlation(merged_df)
41
42     plot_age_groups(merged_df)
43
44     plot_destination_age_pie(merged_df)
45
46
47 if __name__ == '__main__':
48     main()
```

문제 1. 스페이스 타닉 사건 - 구현화면

```
swpilotPython5 — python src/Process4/4-1/main.py — 100x30
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPython5 % python src/Process4/4-1/main.py
[데이터 수량]
총 행 개수 : 12970
총 열 개수 : 14

[상관 분석]

[Transported와의 상관 계수 상위 5개]
FoodCourt      0.046566
ShoppingMall    0.010141
Age            -0.075026
VRDeck         -0.207075
Spa            -0.221131
Name: Transported, dtype: float64
/Users/hwangmungyu/Desktop/swpilotPython5/src/Process4/4-1/main.py:37: FutureWarning: The default of
observed=False is deprecated and will be changed to True in a future version of pandas. Pass observe
d=False to retain current behavior or observed=True to adopt the future default and silence this war
ning.
age_group_transport = train_df.groupby('AgeGroup')['Transported'].mean()
█
```



문제 2. 그때 지구 그리고 한국에서는 - 코드

```
1 # 과정 4 - (문제2) "그때 지구 그리고 한국에서는"
2
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import matplotlib
6
7 matplotlib.rc('font', family='AppleGothic')
8 plt.rcParams['axes.unicode_minus'] = False
9
10 def load_and_filter(filepath):
11     df = pd.read_csv(filepath, encoding='utf-8')
12     df = df[['시점', '성별', '연령별', '일반가구원']]
13     return df
14
15 def group_by_gender_year(df):
16     return df.groupby(['시점', '성별'])['일반가구원'].sum().unstack()
17
18 def group_by_age_year(df):
19     return df.groupby(['시점', '연령별'])['일반가구원'].sum().unstack()
20
21 def plot_gender_age_trend(df):
22     df = df[
23         (df['성별'] != '계') &
24         (~df['연령별'].isin(['합계', '15~64세', '65세이상', '15세미만', '85세이상']))
25     ]
26
27     genders = df['성별'].unique()
28
29     for gender in genders:
30         plt.figure(figsize=(14, 7))
31         gender_df = df[df['성별'] == gender]
32         pivot = gender_df.groupby(['시점', '연령별'])['일반가구원'].sum().unstack()
33
34         y_max = pivot.max().max()
35         plt.ylim(0, y_max * 1.1)
36
37         for age in pivot.columns:
38             plt.plot(pivot.index, pivot[age], marker='o', label=age)
39
40         plt.title(f'{gender} 연령별 일반가구원 통계 (2015년 이후)', fontsize=14)
41         plt.xlabel('년도')
42         plt.ylabel('인원 수')
43         plt.xticks(rotation=45)
44         plt.legend(title='연령대', fontsize=8, ncol=2)
45         plt.grid(True)
46         plt.show()
```

```
1 # [보너스 과제] - 연령별 그래프의 변화를 보고 인구의 변화 트렌드를 데이터로 정리한 리포터 작성
2 def write_markdown_trend(df):
3     df = df[
4         (~df['연령별'].isin(['합계', '15~64세', '65세이상', '15세미만', '85세이상']))
5     ]
6
7     for gender in df['성별'].unique():
8         filename = f'src/Process4/4-2/{gender}_연령별_변화.md'
9         with open(filename, 'w', encoding='utf-8') as f:
10             f.write(f'## {gender} 연령별 인구 변화 트렌드\n\n')
11             gender_df = df[df['성별'] == gender]
12             pivot = gender_df.groupby(['시점', '연령별'])['일반가구원'].sum().unstack()
13
14             for age in pivot.columns:
15                 values = pivot[age].dropna()
16                 start = values.iloc[0]
17                 end = values.iloc[-1]
18                 trend = '증가' if end > start else '감소' if end < start else '유지'
19                 rate = ((end - start) / start) * 100 if start != 0 else 0
20
21                 f.write(f'## {age}\n\n')
22                 f.write(f'- {values.index[0]}년: {int(start):,}명 → {values.index[-1]}년: {int(end):,}명\n\n')
23                 f.write(f'- 변화율: {rate:.1f}% ({trend})\n\n')
24
25 def main():
26     filepath = 'src/Process4/4-2/population_statistics.csv'
27     df = load_and_filter(filepath)
28
29     print('\n 성별 연도별 일반가구원 통계:')
30     gender_year_df = group_by_gender_year(df)
31     print(gender_year_df)
32
33     print('\n 연령별 연도별 일반가구원 통계:')
34     age_year_df = group_by_age_year(df)
35     print(age_year_df)
36
37     plot_gender_age_trend(df)
38     write_markdown_trend(df)
39
40
41 if __name__ == '__main__':
42     main()
```

문제 2. 그때 지구 그리고 한국에서는 - 구현화면

swpilotPython5 — python src/Process4/4-2/main.py — 128x36

```
((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPython5 % python src/Process4/4-2/main.py
```

성별 연도별 일반가구원 통계 :

성별	연도	15~19세	20~24세	25~29세	30~34세	35~39세	40~44세	45~49세	50~54세	55~59세	60~64세	65~69세	70~74세	75~79세	80~84세	85세 이상	합계
남자	2015	138130862	67858092	70271970	70271970	70271970	70271970	70271970	70271970	70271970	70271970	70271970	70271970	70271970	70271970	70271970	70271970
여자	2015	138900968	68274557	70626411	70626411	70626411	70626411	70626411	70626411	70626411	70626411	70626411	70626411	70626411	70626411	70626411	70626411
합계	2015	277031830	136132649	140946181	140946181	140946181	140946181	140946181	140946181	140946181	140946181	140946181	140946181	140946181	140946181	140946181	140946181
남자	2016	139230838	68514490	70716348	70716348	70716348	70716348	70716348	70716348	70716348	70716348	70716348	70716348	70716348	70716348	70716348	70716348
여자	2016	139528824	68705844	70822980	70822980	70822980	70822980	70822980	70822980	70822980	70822980	70822980	70822980	70822980	70822980	70822980	70822980
합계	2016	278759662	137220334	141592328	141592328	141592328	141592328	141592328	141592328	141592328	141592328	141592328	141592328	141592328	141592328	141592328	141592328
남자	2017	139725883	68851790	70874093	70874093	70874093	70874093	70874093	70874093	70874093	70874093	70874093	70874093	70874093	70874093	70874093	70874093
여자	2017	140926459	69442245	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214
합계	2017	280652342	138264235	142325107	142325107	142325107	142325107	142325107	142325107	142325107	142325107	142325107	142325107	142325107	142325107	142325107	142325107
남자	2018	141198880	69733837	71465043	71465043	71465043	71465043	71465043	71465043	71465043	71465043	71465043	71465043	71465043	71465043	71465043	71465043
여자	2018	140892517	69518546	71373971	71373971	71373971	71373971	71373971	71373971	71373971	71373971	71373971	71373971	71373971	71373971	71373971	71373971
합계	2018	282091397	139303883	142834414	142834414	142834414	142834414	142834414	142834414	142834414	142834414	142834414	142834414	142834414	142834414	142834414	142834414
남자	2019	141984847	69647261	71437586	71437586	71437586	71437586	71437586	71437586	71437586	71437586	71437586	71437586	71437586	71437586	71437586	71437586
여자	2019	142834414	70271970	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214	71484214
합계	2019	284819261	140549670	142927000	142927000	142927000	142927000	142927000	142927000	142927000	142927000	142927000	142927000	142927000	142927000	142927000	142927000

연령별 연도별 일반가구원 통계 :

연령대	15~19세	20~24세	25~29세	30~34세	35~39세	40~44세	45~49세	50~54세	55~59세	60~64세	65~69세	70~74세	75~79세	80~84세	85세 이상	합계
2015	5670826	70083986	13777230	5527444	5944874	3485092	2655694	1546708	923060	96679118	138130862	67858092	70271970	70271970	70271970	70271970
2016	5430810	70378900	13506152	5593146	6040326	3439550	2778352	1678316	985954	97102696	139230838	68514490	70716348	70716348	70716348	70716348
2017	5146408	70222416	13227766	5501480	6218248	3452280	2997558	1767350	1029222	97229814	139725883	68851790	70874093	70874093	70874093	70874093
2018	4820648	70118948	12931884	5444456	6408732	3593212	3073860	1876074	1098258	97329844	140926459	69442245	71484214	71484214	71484214	71484214
2019	4524240	69825528	12593296	5352254	6569438	3776278	3089832	2002348	1200096	97348354	141198880	69733837	71465043	71465043	71465043	71465043
2020	4417504	70020460	12319444	5580930	6760114	3960150	3081740	2064094	1291546	98057454	140892517	69518546	71373971	71373971	71373971	71373971
2021	4180092	69588650	11983426	5341198	6801484	4084308	3059980	2184734	1421268	98127862	141984847	69647261	71437586	71437586	71437586	71437586
2022	4089452	68762104	11522842	4852182	6400344	4209236	3183580	2403294	1554864	97769292	142834414	70271970	71484214	71484214	71484214	71484214
2023	4083358	68309856	11053294	4612138	6538600	4405582	3248660	2481234	1678900	97740996	142927000	70874093	71484214	71484214	71484214	71484214

[9 rows x 19 columns]

src > Process4 > 4-2 > # 계 연령별 인구 변화 트렌드

계 연령별 인구 변화 트렌드

1 15~19세

2 20~24세

3 25~29세

4 30~34세

5 35~39세

6 40~44세

7 45~49세

8 50~54세

9 55~59세

10 60~64세

11 65~69세

12 70~74세

13 75~79세

14 80~84세

15 85세 이상

16 합계

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

계 연령별 인구 변화 트렌드

15~19세

- 2015년: 2,835,413명 → 2023년: 2,041,679명
- 변화율: -28.0% (감소)

20~24세

- 2015년: 2,763,722명 → 2023년: 2,306,069명
- 변화율: -16.6% (감소)

25~29세

- 2015년: 2,972,437명 → 2023년: 3,269,300명
- 변화율: 10.0% (증가)

30~34세

- 2015년: 3,570,246명 → 2023년: 3,281,148명
- 변화율: -8.1% (감소)

35~39세

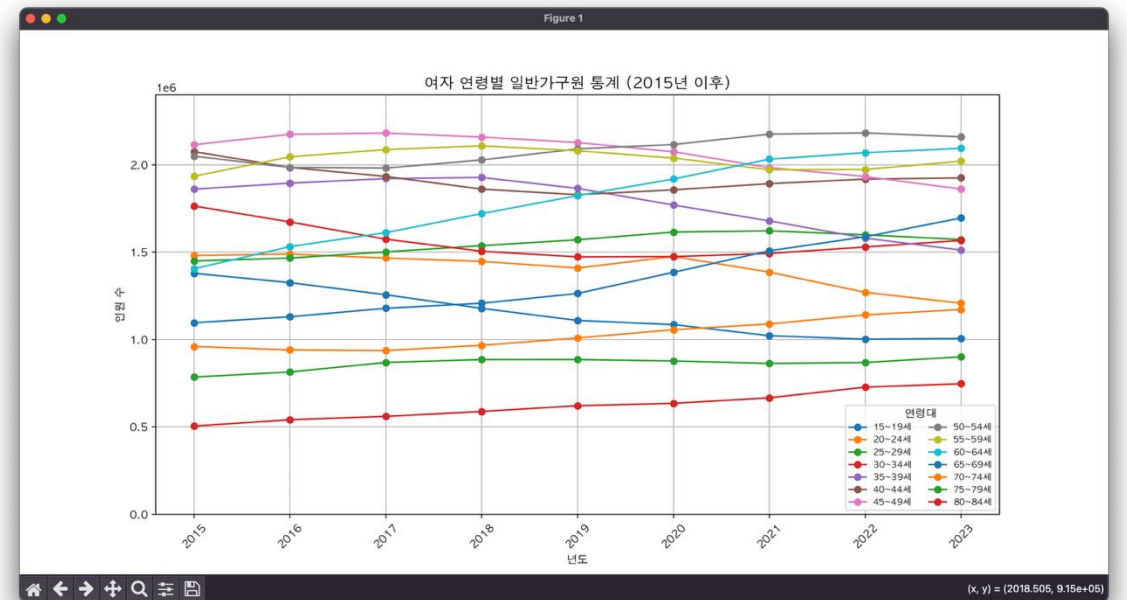
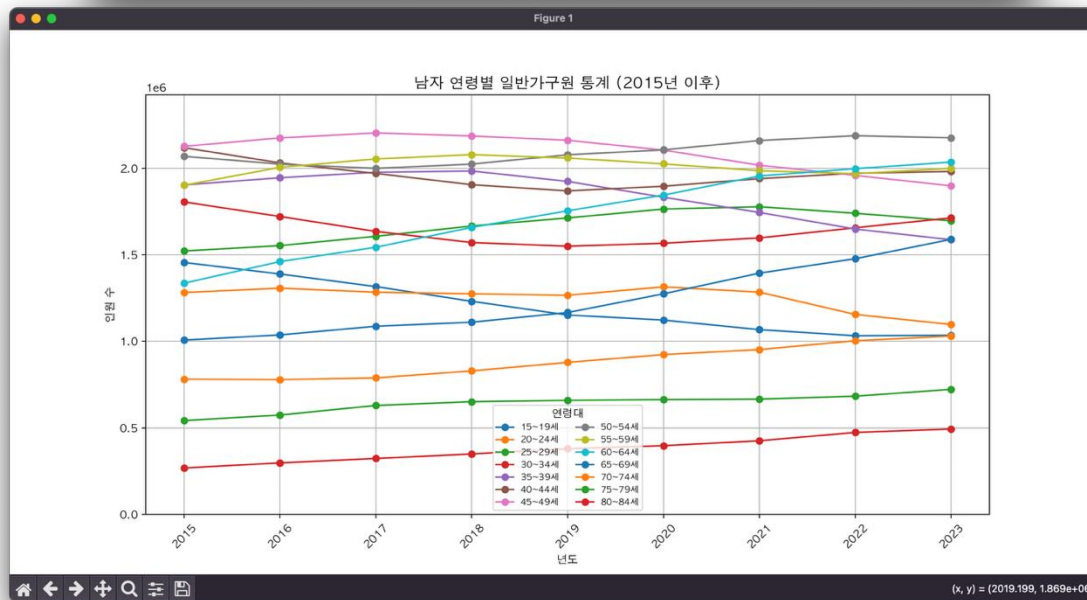
- 2015년: 3,765,009명 → 2023년: 3,099,533명
- 변화율: -17.7% (감소)

40~44세

- 2015년: 4,119,181명 → 2023년: 4,336,866명
- 변화율: 5.3% (증가)

45~49세

- 2015년: 3,837,143명 → 2023년: 4,021,368명
- 변화율: 4.8% (증가)



문제 3. 스마트 팜의 시작 - 코드

```
1 # 과정 4 - (문제3) "스마트 팜의 시작"
2
3 import threading
4 import time
5 import random
6 from datetime import datetime
7 import pandas as pd
8
9 # 데이터 저장용 리스트
10 sensor_log = []
11
12 # 출력 꼬임 방지 lock
13 print_lock = threading.Lock()
14
15 # 스마트팜 센서 클래스
16 class ParmSensor:
17     def __init__(self, name):
18         self.name = name
19         self.temperature = 0
20         self.light = 0
21         self.humidity = 0
22
23     def set_data(self):
24         self.temperature = random.randint(20, 30)
25         self.light = random.randint(5000, 10000)
26         self.humidity = random.randint(40, 70)
27
28     def get_data(self):
29         return self.temperature, self.light, self.humidity
30
31 # 센서 작동 쓰레드
32 def sensor_thread(sensor):
33     while True:
34         sensor.set_data()
35         temp, light, humi = sensor.get_data()
36         timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
37         log = {
38             'timestamp': timestamp,
39             'sensor': sensor.name,
40             'temperature': temp,
41             'light': light,
42             'humidity': humi
43         }
44         sensor_log.append(log)
45         with print_lock:
46             print(f'{timestamp} {sensor.name} - temp {temp}, light {light}, humi {humi}')
47         time.sleep(10)
```

```
1 def average_logger():
2     while True:
3         time.sleep(300)
4         if not sensor_log:
5             with print_lock:
6                 print('\n[5분 단위 평균] 데이터 없음\n')
7             return
8
9     df = pd.DataFrame(sensor_log)
10    df['timestamp'] = pd.to_datetime(df['timestamp'])
11    df.set_index('timestamp', inplace=True)
12
13    numeric_cols = ['temperature', 'light', 'humidity']
14    resampled = df[numeric_cols].resample('5min').mean().round(2)
15
16    current_time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
17    with print_lock:
18        print(f'\n[누적 데이터의 5분 단위 평균 - {current_time} 기준]')
19        print('온도'.rjust(6), '조도'.rjust(10), '습도'.rjust(6))
20        print('-' * 30)
21        for row in resampled.itertuples(index=False):
22            print(f'{row.temperature:^6} {row.light:^10} {row.humidity:^6}')
23        print()
24
25 def main():
26     sensors = [ParmSensor(f'Parm-{i}') for i in range(1, 6)]
27
28     for sensor in sensors:
29         t = threading.Thread(target=sensor_thread, args=(sensor,))
30         t.daemon = True
31         t.start()
32
33     avg_thread = threading.Thread(target=average_logger)
34     avg_thread.daemon = True
35     avg_thread.start()
36
37     # 메인 쓰레드 대기
38     while True:
39         time.sleep(1)
40
41 if __name__ == '__main__':
42     main()
```

문제 3. 스마트 팜의 시작 - 구현화면

```
swpilotPython5 — python src/Process4/4-3/main.py — 112x30
2025-05-27 11:36:03 Parm-5 — temp 25, light 8619, humi 68
2025-05-27 11:36:03 Parm-2 — temp 24, light 6164, humi 61
2025-05-27 11:36:03 Parm-4 — temp 30, light 8593, humi 62
2025-05-27 11:36:03 Parm-1 — temp 20, light 5527, humi 43
2025-05-27 11:36:13 Parm-5 — temp 30, light 8025, humi 56
2025-05-27 11:36:13 Parm-3 — temp 25, light 8074, humi 61
2025-05-27 11:36:13 Parm-2 — temp 28, light 8201, humi 53
2025-05-27 11:36:13 Parm-4 — temp 28, light 7516, humi 46
2025-05-27 11:36:13 Parm-1 — temp 26, light 6955, humi 50

[누적 데이터의 5분 단위 평균 - 2025-05-27 11:36:23 기준]
온도          조도          습도
-----
25.38      7330.35   54.78
2025-05-27 11:36:23 Parm-5 — temp 22, light 5630, humi 69
2025-05-27 11:36:23 Parm-2 — temp 21, light 5759, humi 70
2025-05-27 11:36:23 Parm-3 — temp 20, light 6246, humi 62
2025-05-27 11:36:23 Parm-4 — temp 26, light 5010, humi 44
2025-05-27 11:36:23 Parm-1 — temp 30, light 5011, humi 42
2025-05-27 11:36:33 Parm-2 — temp 21, light 8014, humi 52
2025-05-27 11:36:33 Parm-5 — temp 29, light 6748, humi 47
2025-05-27 11:36:33 Parm-3 — temp 24, light 5458, humi 54
2025-05-27 11:36:33 Parm-1 — temp 20, light 7532, humi 57
2025-05-27 11:36:33 Parm-4 — temp 29, light 5730, humi 41
2025-05-27 11:36:43 Parm-5 — temp 26, light 6351, humi 53
2025-05-27 11:36:43 Parm-2 — temp 27, light 8688, humi 53
2025-05-27 11:36:43 Parm-3 — temp 26, light 9056, humi 62
2025-05-27 11:36:43 Parm-1 — temp 22, light 9791, humi 66
2025-05-27 11:36:43 Parm-4 — temp 24, light 6575, humi 42
```

문제 4. 스마트 팜 데이터의 저장과 활용 - 코드

```
1 # 과정 4 - (문제4) "스마트 팜 데이터의 저장과 활용"
2
3 import threading
4 import time
5 import random
6 from datetime import datetime
7 import mysql.connector
8 import os
9
10 sensor_log = []
11 print_lock = threading.Lock()
12
13 class ParmSensor:
14     def __init__(self, name):
15         self.name = name
16         self.temperature = 0
17         self.light = 0
18         self.humidity = 0
19
20     def set_data(self):
21         self.temperature = random.randint(20, 30)
22         self.light = random.randint(5000, 10000)
23         self.humidity = random.randint(40, 70)
24
25     def get_data(self):
26         return self.temperature, self.light, self.humidity
27
28 def insert_sensor_data(sensor_name, temperature, light, humidity):
29     try:
30         conn = mysql.connector.connect(
31             host=os.getenv('MYSQL_HOST', 'localhost'),
32             user=os.getenv('MYSQL_USER', 'root'),
33             password=os.getenv('MYSQL_PASSWORD', ''),
34             database=os.getenv('MYSQL_DATABASE', 'swpilot')
35         )
36         cursor = conn.cursor()
37         query = '''
38             INSERT INTO parm_data (sensor_name, timestamp, temperature, light, humidity)
39             VALUES (%s, %s, %s, %s, %s)
40             '''
```

```
1         now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
2         cursor.execute(query, (sensor_name, now, temperature, light, humidity))
3         conn.commit()
4         cursor.close()
5         conn.close()
6     except Exception as e:
7         with print_lock:
8             print(f'[DB 오류] {sensor_name} → {e}')
9
10 def sensor_worker(sensor):
11     while True:
12
13         sensor.set_data()
14         temp, light, humi = sensor.get_data()
15         timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
16
17         with print_lock:
18             print(f'{timestamp} {sensor.name} - temp {temp}, light {light}, humi {humi}')
19             sensor_log.append({
20                 'sensor_name': sensor.name,
21                 'timestamp': timestamp,
22                 'temperature': temp,
23                 'light': light,
24                 'humidity': humi
25             })
26             insert_sensor_data(sensor.name, temp, light, humi)
27
28         time.sleep(10)
29
30 def start_sensors():
31     sensors = [ParmSensor(f'Parm-{i}') for i in range(1, 6)]
32     for s in sensors:
33         t = threading.Thread(target=sensor_worker, args=(s,))
34         t.daemon = True
35         t.start()
36
37 def main():
38     start_sensors()
39     while True:
40         time.sleep(1) # 메인 스레드 유지용
41
42 if __name__ == '__main__':
43     main()
```


문제 4. 스마트 팜 데이터의 저장과 활용 - 구현화면

```
1 USE swpilot;
2 CREATE TABLE parm_data (
3   id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
4   sensor_name VARCHAR(20) NOT NULL,
5   timestamp DATETIME NOT NULL,
6   temperature INT NOT NULL,
7   light INT NOT NULL,
8   humidity INT NOT NULL
9 );
10
11 DESC parm_data;
```

Field	Type	Null	Key	Default	Extra
id	int unsigned	NO	PRI		auto_increment
sensor_name	varchar(20)	NO			
timestamp	datetime	NO			
temperature	int	NO			
light	int	NO			
humidity	int	NO			

Result 4

Time	Action	Response	Duration / Fetch Time
11:45:15	USE swpilot	0 row(s) affected	0.00057 sec
11:45:15	CREATE TABLE parm_data (id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, sensor_name VARCHAR(20) NOT NULL, timestamp DATETIME NOT NULL, temperature INT NOT NULL, light INT NOT NULL, humidity INT NOT NULL)	0 row(s) affected	0.0073 sec
11:45:15	DESC parm_data	6 row(s) returned	0.0032 sec / 0.00001...

```
hwangmungyu — mysql -u root -p — 100x30

[mysql> USE swpilot;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
[mysql> SELECT * FROM parm_data ORDER BY id DESC
-> ;
```

id	sensor_name	timestamp	temperature	light	humidity
15	Parm-5	2025-05-29 09:18:47	29	7898	49
14	Parm-4	2025-05-29 09:18:47	28	9383	61
13	Parm-3	2025-05-29 09:18:47	30	8011	44
12	Parm-2	2025-05-29 09:18:47	25	5792	56
11	Parm-1	2025-05-29 09:18:47	23	8710	63
10	Parm-5	2025-05-29 09:18:37	24	5135	68
9	Parm-4	2025-05-29 09:18:37	21	5600	44
8	Parm-3	2025-05-29 09:18:37	20	9426	65
7	Parm-2	2025-05-29 09:18:37	20	8766	57
6	Parm-1	2025-05-29 09:18:37	27	8922	52
5	Parm-5	2025-05-29 09:18:27	26	5985	53
4	Parm-4	2025-05-29 09:18:27	24	6787	48
3	Parm-3	2025-05-29 09:18:27	24	6098	61
2	Parm-2	2025-05-29 09:18:27	26	7920	63
1	Parm-1	2025-05-29 09:18:27	27	8691	42

```
mysql> ]
15 rows in set (0.00 sec)

mysql> ]
```

```
mysql> SELECT sensor_name, COUNT(*) AS record_count
-> FROM parm_data
-> GROUP BY sensor_name;
```

sensor_name	record_count
Parm-1	3
Parm-2	3
Parm-3	3
Parm-4	3
Parm-5	3

```
mysql> ]
5 rows in set (0.01 sec)
```

```
hwangmungyu — mysql -u root -p — 100x30

mysql> SELECT
-> FROM_UNIXTIME(FLOOR(UNIX_TIMESTAMP(timestamp) / 10) * 10) AS time_block,
-> ROUND(AVG(temperature), 2) AS avg_temp,
-> ROUND(AVG(light), 2) AS avg_light,
-> ROUND(AVG(humidity), 2) AS avg_humi
-> FROM parm_data
-> GROUP BY time_block
-> ORDER BY time_block;
```

time_block	avg_temp	avg_light	avg_humi
2025-05-29 09:18:20	25.40	7096.20	53.40
2025-05-29 09:18:30	22.40	7569.80	57.20
2025-05-29 09:18:40	27.00	7958.80	54.60

```
mysql> ]
3 rows in set (0.00 sec)
```

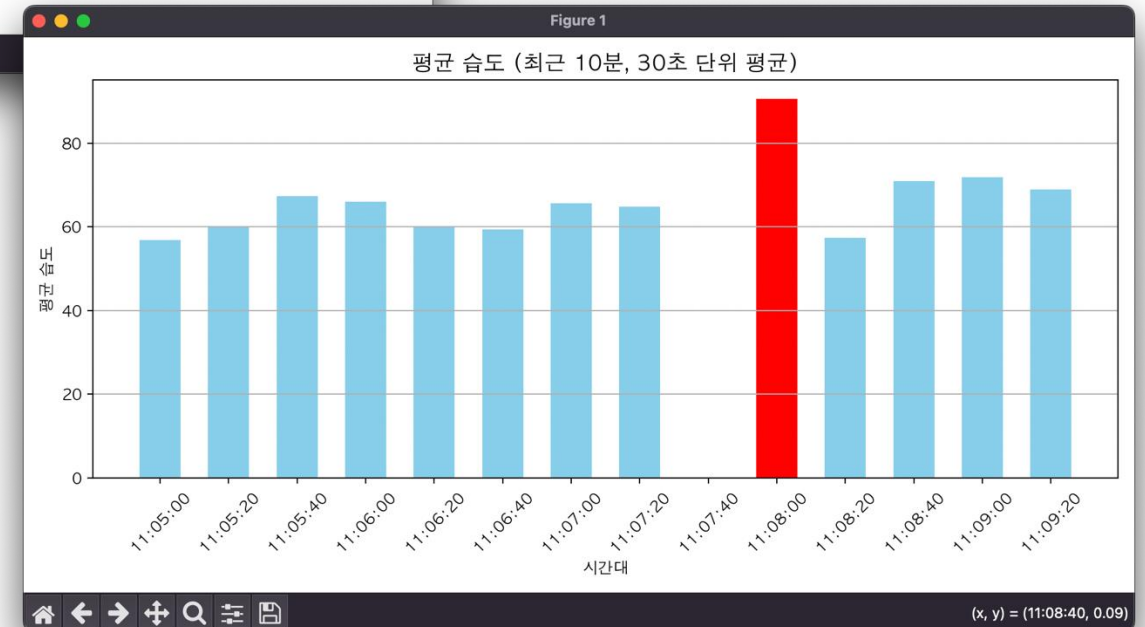
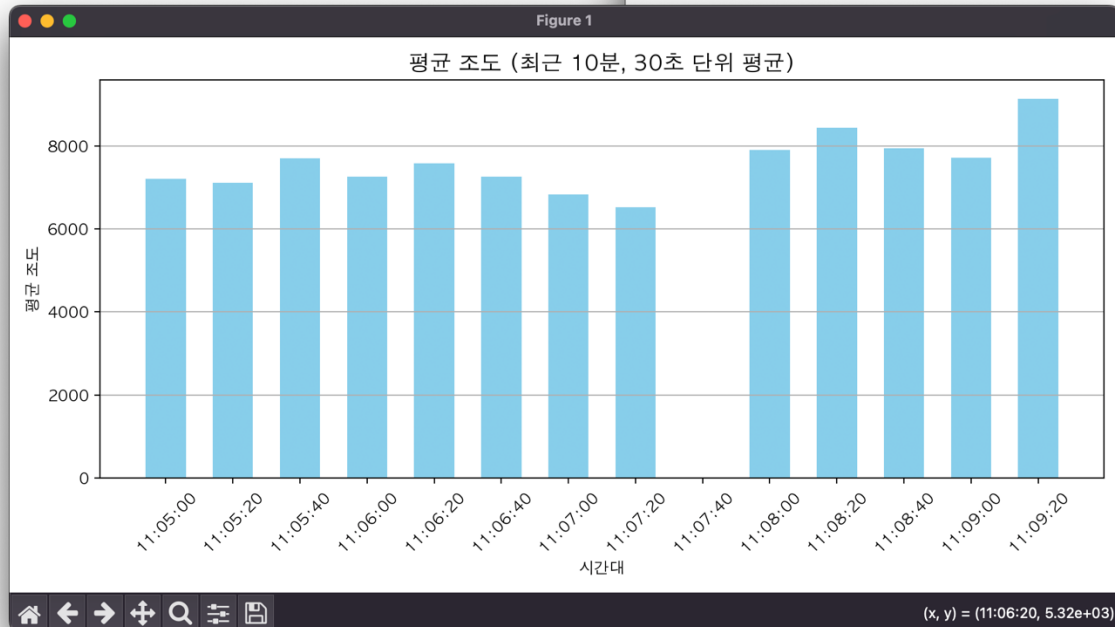
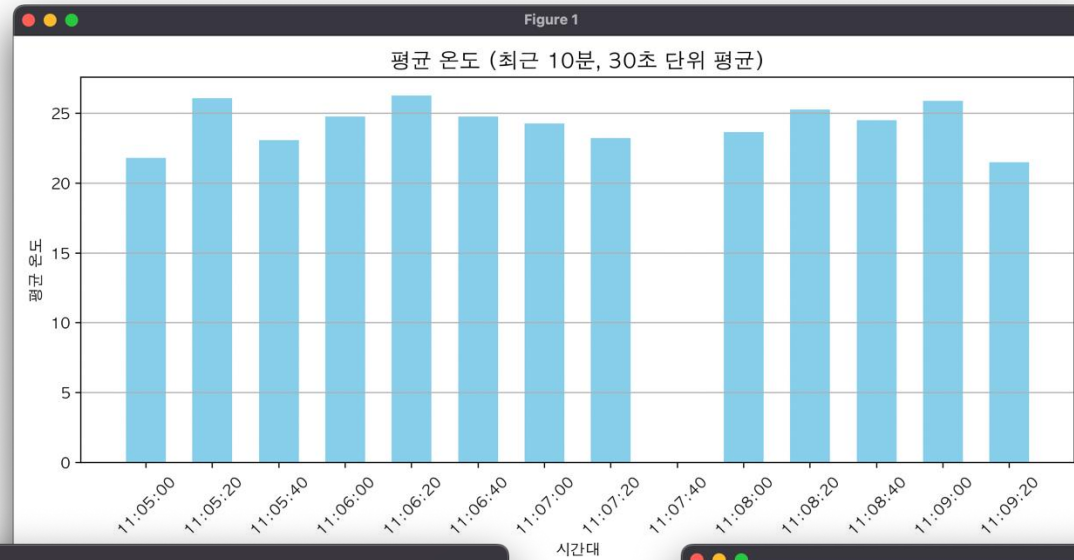
문제 5. 중간 지점으로 큐를 사용해 볼까? - 코드

```
1 # 과제 4 - (문제5) "중간 지점으로 큐를 사용해 볼까?"
2
3 import threading
4 import random
5 import time
6 from datetime import datetime
7 from queue import Queue
8 import os
9 import mysql.connector
10 import pandas as pd
11 import matplotlib.pyplot as plt
12 from matplotlib.dates import DateFormatter
13 import matplotlib
14
15 matplotlib.rc('font', family='AppleGothic')
16 plt.rcParams['axes.unicode_minus'] = False
17
18 print_lock = threading.Lock()
19
20 class ParmSensor:
21     def __init__(self, name):
22         self.name = name
23
24     def set_data(self):
25         return {
26             'sensor_name': self.name,
27             'timestamp': datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
28             'temperature': random.randint(20, 30),
29             'light': random.randint(5000, 10000),
30             'humidity': random.randint(40, 90)
31         }
32
33 def insert_sensor_data(data):
34     conn = mysql.connector.connect(
35         host=os.getenv('MYSQL_HOST', 'localhost'),
36         user=os.getenv('MYSQL_USER', 'root'),
37         password=os.getenv('MYSQL_PASSWORD', ''),
38         database=os.getenv('MYSQL_DATABASE', 'swpilot')
39     )
40     cursor = conn.cursor()
41     query = '''
42         INSERT INTO parm_data (sensor_name, timestamp, temperature, light, humidity)
43         VALUES (%s, %s, %s, %s, %s)
44     '''
45     cursor.execute(query, (
46         data['sensor_name'],
47         data['timestamp'],
48         data['temperature'],
49         data['light'],
50         data['humidity']
51     ))
52     conn.commit()
53     cursor.close()
54     conn.close()
```

```
1 def sensor_thread(sensor, queue):
2     while True:
3         data = sensor.set_data()
4         with print_lock:
5             print(f"data['timestamp']: {data['sensor_name']} - temp {data['temperature']}, light {data['light']}, humi {data['humidity']}")
6             queue.put(data)
7         time.sleep(10)
8
9 def db_writer_thread(queue):
10     while True:
11         if not queue.empty():
12             with print_lock:
13                 data = queue.get()
14                 insert_sensor_data(data)
15             time.sleep(1)
16
17 def get_sensor_data():
18     conn = mysql.connector.connect(
19         host=os.getenv('MYSQL_HOST', 'localhost'),
20         user=os.getenv('MYSQL_USER', 'root'),
21         password=os.getenv('MYSQL_PASSWORD', ''),
22         database=os.getenv('MYSQL_DATABASE', 'swpilot')
23     )
24     # 최근 10분 데이터
25     query = '''
26         SELECT * FROM parm_data
27         WHERE timestamp >= NOW() - INTERVAL 10 MINUTE
28     '''
29     df = pd.read_sql(query, conn)
30     conn.close()
31     return df
32
33 def visualize_timeblock_averages_from_df(df):
34     df['timestamp'] = pd.to_datetime(df['timestamp'])
35     df.set_index('timestamp', inplace=True)
36
37     numeric_df = df[['temperature', 'light', 'humidity']]
38
39     avg_df = numeric_df.resample('20s').mean().round(2).reset_index()
40
41     avg_df['time_label'] = avg_df['timestamp'].dt.strftime('%H:%M:%S')
42
43     metrics = {
44         'temperature': '평균 온도',
45         'light': '평균 조도',
46         'humidity': '평균 습도'
47     }
48
49     return avg_df, metrics
```

```
1 for col, label in metrics.items():
2     plt.figure(figsize=(10, 5))
3     # [보너스 과제] - # 습도 90% 이상은 빨간색, 그 외는 파란색
4     if col == 'humidity':
5         colors = ['red' if val > 90 else 'skyblue' for val in avg_df[col]]
6     else:
7         colors = 'skyblue'
8     plt.bar(avg_df['time_label'], avg_df[col], width=0.6, align='center', color=colors)
9     plt.title(f'{label} (최근 10분, 30초 단위 평균)', fontsize=14)
10    plt.xlabel('시간대')
11    plt.ylabel(label)
12    plt.xticks(rotation=45)
13    plt.grid(True, axis='y')
14    plt.tight_layout()
15    plt.show()
16
17 def main():
18     # 큐 및 센서 초기화
19     sensor_q = Queue()
20     sensors = [ParmSensor(f'Parm-{i}') for i in range(1, 6)]
21
22     # 센서 데이터 생성 스레드 시작
23     for s in sensors:
24         threading.Thread(target=sensor_thread, args=(s, sensor_q), daemon=True).start()
25
26     # DB 저장 스레드 시작
27     threading.Thread(target=db_writer_thread, args=(sensor_q,), daemon=True).start()
28
29     # 메인 루프 유지 + 주기적으로 시각화 호출 (예: 60초마다)
30     try:
31         while True:
32             time.sleep(60)
33             df = get_sensor_data()
34             visualize_timeblock_averages_from_df(df)
35     except KeyboardInterrupt:
36         print('\n[종료] 사용자가 프로그램을 중단했습니다.')
37
38 if __name__ == '__main__':
39     main()
```

문제 5. 중간 지점으로 큐를 사용해 볼까? - 구현화면

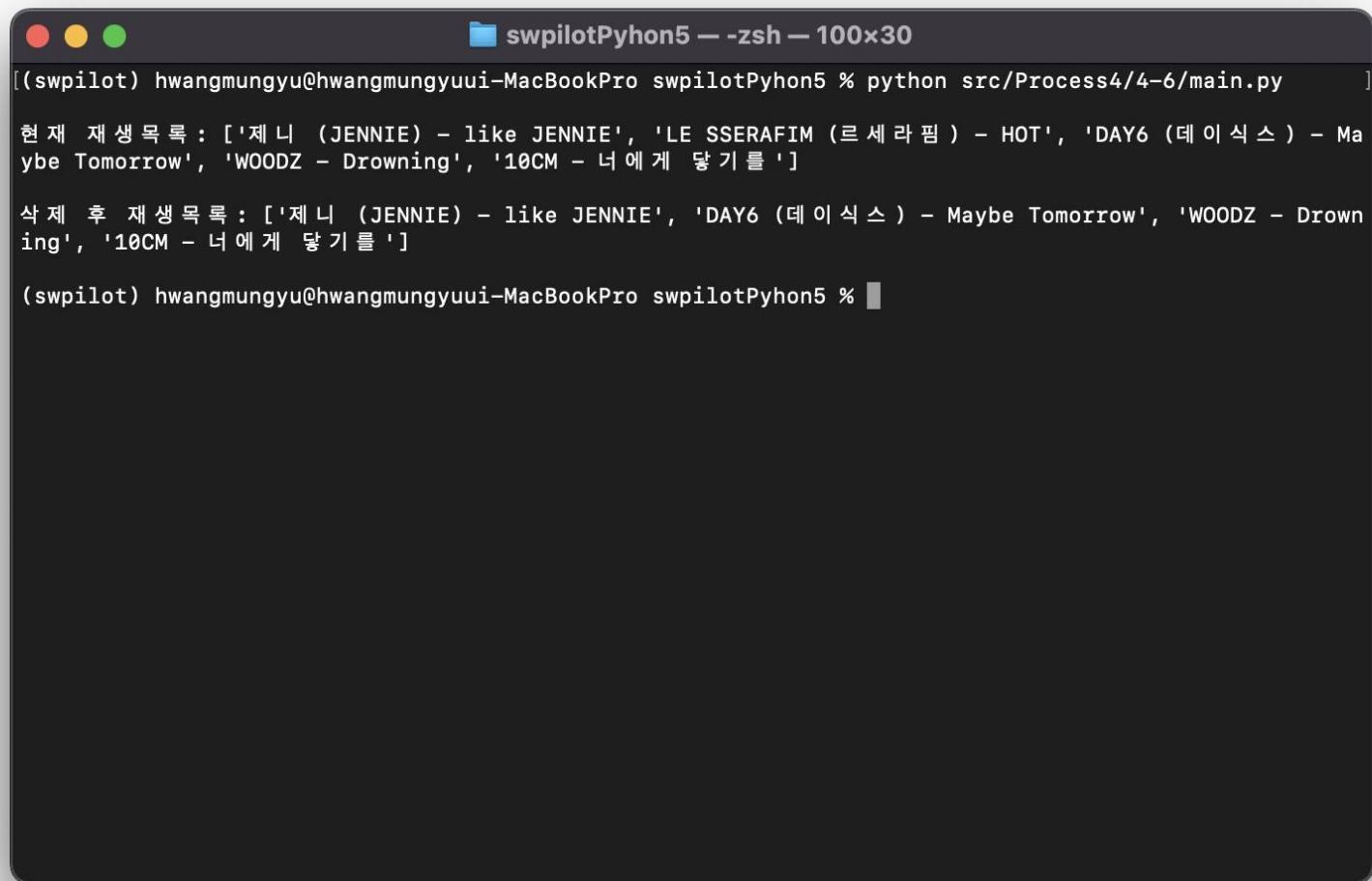


문제 6. 음악이 필요해 - 코드

```
1 # 과정 4 - (문제6) "음악이 필요해"
2
3 class Node:
4     def __init__(self, data):
5         self.data = data
6         self.next = None
7
8 class LinkedList:
9     def __init__(self):
10        self.head = None
11
12    def insert(self, data, position=None):
13        new_node = Node(data)
14
15        # 1. 빈 리스트인 경우
16        if self.head is None:
17            self.head = new_node
18            return
19
20        # 2. 맨 앞에 삽입
21        if position == 0:
22            new_node.next = self.head
23            self.head = new_node
24            return
25
26        # 3. 중간 또는 맨 뒤 삽입
27        current = self.head
28        index = 0
29        prev = None
30        while current and (position is None or index < position):
31            prev = current
32            current = current.next
33            index += 1
34
35        new_node.next = current
36        if prev:
37            prev.next = new_node
```

```
1     def delete(self, data):
2         current = self.head
3         prev = None
4
5         while current:
6             if current.data == data:
7                 if prev is None:
8                     # 삭제 대상이 head
9                     self.head = current.next
10                else:
11                    prev.next = current.next
12                    return True # 삭제 성공
13                prev = current
14                current = current.next
15
16        return False # 삭제 실패
17
18    def get_list(self):
19        result = []
20        current = self.head
21
22        while current:
23            result.append(current.data)
24            current = current.next
25
26        return result
27
28    # 사용 예시
29    if __name__ == '__main__':
30        playlist = LinkedList()
31
32        # 음악 추가
33        playlist.insert('LE SSERAFIM (르세라핌) - HOT')
34        playlist.insert('DAY6 (데이식스) - Maybe Tomorrow')
35        playlist.insert('WOODZ - Drowning', 2) # 중간 삽입
36        playlist.insert('제니 (JENNIE) - like JENNIE', 0) # 맨 앞
37        playlist.insert('10CM - 너에게 당기를')
38
39        # 현재 목록 출력
40        print('\n현재 재생목록:', playlist.get_list())
41
42        # 음악 삭제
43        playlist.delete('LE SSERAFIM (르세라핌) - HOT')
44
45        print(f'\n삭제 후 재생목록: {playlist.get_list()}\n')
```

문제 6. 음악이 필요해 - 구현화면



```
swpilotPyhon5 — -zsh — 100x30
((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process4/4-6/main.py
]
현재 재생 목록 : ['제니 (JENNIE) - like JENNIE', 'LE SSERAFIM (르세라핌) - HOT', 'DAY6 (데이식스) - Maybe Tomorrow', 'WOODZ - Drowning', '10CM - 너에게 닿기를']

삭제 후 재생 목록 : ['제니 (JENNIE) - like JENNIE', 'DAY6 (데이식스) - Maybe Tomorrow', 'WOODZ - Drowning', '10CM - 너에게 닿기를']

((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 %
```


문제 7. 꿈기지 않는 음악 - 코드

```
1 # 과장 4 - (문제7) "꿈기지 않는 음악"
2
3 import os
4 import time
5
6 class Node:
7     def __init__(self, data):
8         self.data = data
9         self.next = None
10
11 class CircularLinkedList:
12     def __init__(self):
13         self.tail = None # 마지막 노드를 가리킴
14         self.current = None # get_next를 포인터
15
16     def insert(self, data):
17         new_node = Node(data)
18         if self.tail is None:
19             self.tail = new_node
20             new_node.next = new_node
21             self.current = new_node
22         else:
23             new_node.next = self.tail.next
24             self.tail.next = new_node
25             self.tail = new_node
26
27     def delete(self, data):
28         if self.tail is None:
29             print('리스트가 비어 있습니다.')
30             return
31
32         prev = self.tail
33         curr = self.tail.next
34
35         while True:
36             if curr.data == data:
37                 if curr == self.tail:
38                     if curr.next == curr:
39                         self.tail = None
40                         self.current = None
41                     else:
42                         prev.next = curr.next
43                         self.tail = prev
44                 else:
45                     prev.next = curr.next
46                     print(f'{data} 삭제 완료')
47                     return
48                 prev, curr = curr, curr.next
49                 if curr == self.tail.next:
50                     break
51             print(f'{data}를 찾을 수 없습니다.')
52
53     def get_next(self):
54         if self.current is None:
55             print('리스트가 비어 있습니다.')
56             return None
57         self.current = self.current.next
58         return self.current.data
59
```

```
1 def search(self, keyword):
2     if self.tail is None:
3         return []
4
5     result = []
6     curr = self.tail.next
7     while True:
8         if keyword.lower() in curr.data.lower():
9             result.append(curr.data)
10            curr = curr.next
11            if curr == self.tail.next:
12                break
13        return result
14
15 def display(self, count=10):
16     if self.tail is None:
17         print('빈 리스트')
18         return
19
20     curr = self.tail.next
21     for _ in range(count):
22         print(curr.data)
23         curr = curr.next
24
25
26 if __name__ == '__main__':
27     circularlist = CircularLinkedList()
28
29     # 예시: 일본 인기곡 6곡
30     songs = [
31         '10CM - 난에게 닿기를',
32         '제니 (JENNIE) - like JENNIE',
33         'WOODZ - Drowning',
34         'DAY6 - Maybe Tomorrow',
35         'aespa - Whiplash',
36         'LE SSERAFIM - HOT'
37     ]
38
39     for song in songs:
40         circularlist.insert(song)
41
42     print()
43     for _ in range(10):
44         print('>', circularlist.get_next())
45         time.sleep(1)
46
47     print('\n"JENNIE" 검색 결과:')
48     for result in circularlist.search('JENNIE'):
49         print('>', result)
50
51     print('\nWOODZ - Drowning 삭제:')
52     circularlist.delete('WOODZ - Drowning')
53
54     print('\n삭제 이후 원형 연결 리스트 상태:')
55     circularlist.display(6)
56     print()
57
58 # # [보너스 과제] - 노래 제목 말고 실제 mp3 등의 파일 경로를 입력하고 순차적으로 음악이 재생하도록 음악 플레이어 상단
59 # from playsound import playsound
60
61 # def play_music_files(circularlist):
62 #     while True:
63 #         song = circularlist.get_next()
64 #         print('.', Now playing:', song)
65 #         playsound(song) # mp3 파일 경로일 경우
66 #         time.sleep(1)
```

문제 7. 꿈기지 않는 음악 - 구현화면

```
swpilotPyhon5 — -zsh — 100x30
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process4/4-7/main.py]

▶ 제 니 (JENNIE) - like JENNIE
▶ WOODZ - Drowning
▶ DAY6 - Maybe Tomorrow
▶ aespa - Whiplash
▶ LE SSERAFIM - HOT
▶ 10CM - 너 에 게 달 기 를
▶ 제 니 (JENNIE) - like JENNIE
▶ WOODZ - Drowning
▶ DAY6 - Maybe Tomorrow
▶ aespa - Whiplash

"JENNIE" 검색 결과 :
▶ 제 니 (JENNIE) - like JENNIE

WOODZ - Drowning 삭제 :
"WOODZ - Drowning" 삭제 완료

삭제 이후 원형 연결 리스트 상태 :
10CM - 너 에 게 달 기 를
제 니 (JENNIE) - like JENNIE
DAY6 - Maybe Tomorrow
aespa - Whiplash
LE SSERAFIM - HOT
10CM - 너 에 게 달 기 를

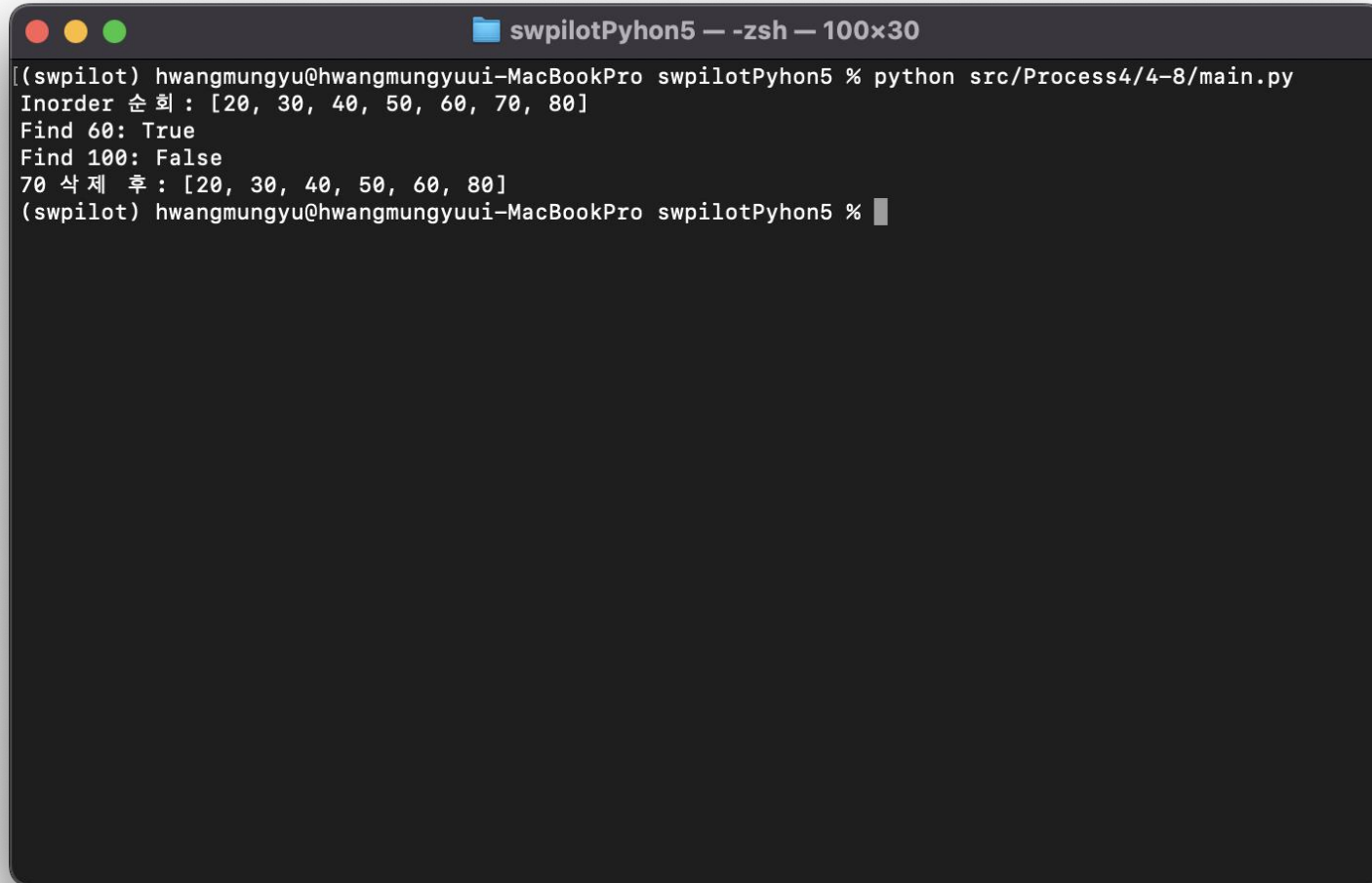
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % ]
```

문제 8. 식물의 계보를 추적하자 - 코드

```
1 # 과정 4 - (문제8) "식물의 계보를 추적하자"
2
3 class Node:
4     def __init__(self, value):
5         self.value = value
6         self.left = None
7         self.right = None
8
9 class BinarySearchTree:
10     def __init__(self):
11         self.root = None
12
13     def insert(self, value):
14         new_node = Node(value)
15         if self.root is None:
16             self.root = new_node
17             return
18
19         current = self.root
20         while True:
21             if value < current.value:
22                 if current.left is None:
23                     current.left = new_node
24                     return
25                 current = current.left
26             else:
27                 if current.right is None:
28                     current.right = new_node
29                     return
30                 current = current.right
31
32     def find(self, value):
33         current = self.root
34         while current is not None:
35             if value == current.value:
36                 return True
37             elif value < current.value:
38                 current = current.left
39             else:
40                 current = current.right
41         return False
42
43     def delete(self, value):
44         self.root = self._delete_node(self.root, value)
```

```
1     def _delete_node(self, node, value):
2         if node is None:
3             return node
4
5         if value < node.value:
6             node.left = self._delete_node(node.left, value)
7         elif value > node.value:
8             node.right = self._delete_node(node.right, value)
9         else:
10             if node.left is None:
11                 return node.right
12             elif node.right is None:
13                 return node.left
14
15             # 양쪽 자식이 있는 경우
16             temp = self._min_value_node(node.right)
17             node.value = temp.value
18             node.right = self._delete_node(node.right, temp.value)
19
20         return node
21
22     def _min_value_node(self, node):
23         while node.left is not None:
24             node = node.left
25         return node
26
27     def inorder(self):
28         result = []
29         self._inorder_traversal(self.root, result)
30         return result
31
32     def _inorder_traversal(self, node, result):
33         if node:
34             self._inorder_traversal(node.left, result)
35             result.append(node.value)
36             self._inorder_traversal(node.right, result)
37
38 # 사용 예시
39 if __name__ == '__main__':
40     tree = BinarySearchTree()
41     values = [50, 30, 70, 20, 40, 60, 80]
42
43     for v in values:
44         tree.insert(v)
45
46     print('Inorder 순회:', tree.inorder()) # [20, 30, 40, 50, 60, 70, 80]
47
48     print('Find 60:', tree.find(60))      # True
49     print('Find 100:', tree.find(100))    # False
50
51     tree.delete(70)
52     print('70 삭제 후:', tree.inorder())  # [20, 30, 40, 50, 60, 80]
```

문제 8. 식물의 계보를 추적하자 - 구현화면



```
swpilotPyhon5 — -zsh — 100x30
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process4/4-8/main.py ]
Inorder 순회 : [20, 30, 40, 50, 60, 70, 80]
Find 60: True
Find 100: False
70 삭제 후 : [20, 30, 40, 50, 60, 80]
(swipilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 %
```

문제 9. 비료 자동 투입기 - 코드

```
1 # 과정 4 - (문제9) "비료 자동 투입기"
2
3 import matplotlib.pyplot as plt
4 import matplotlib
5
6 matplotlib.rc('font', family='AppleGothic')
7 plt.rcParams['axes.unicode_minus'] = False
8
9 class Stack:
10     def __init__(self, max_size=10):
11         self.items = []
12         self.max_size = max_size
13
14     def push(self, item):
15         if len(self.items) >= self.max_size:
16             print('[경고] 스택이 가득 찼습니다. 더 이상 push할 수 없습니다.\n')
17         else:
18             self.items.append(item)
19             print(f'[push] {item} 추가됨\n')
20
21     def pop(self):
22         if self.empty():
23             print('[경고] 스택이 비어있습니다. pop할 수 없습니다.\n')
24             return None
25         else:
26             item = self.items.pop()
27             print(f'[pop] {item} 제거됨\n')
28             return item
29
30     def peek(self):
31         if self.empty():
32             print('[경고] 스택이 비어있습니다. 확인할 수 없습니다.\n')
33             return None
34         else:
35             print(f'[peek] 현재 top 항목은: {self.items[-1]}\n')
36             return self.items[-1]
37
38     def empty(self):
39         return len(self.items) == 0
40
```

```
1 # [보너스 과제] - 스택 구조 클래스화 및 스택의 상태를 시각화
2 def visualize(self):
3     print('\n[Stack 시각화]')
4     fig, ax = plt.subplots(figsize=(6, 6))
5     ax.set_xlim(0, 1)
6     ax.set_ylim(0, 1)
7     ax.axis('off')
8
9     box_height = 1 / self.max_size
10
11     for i in range(self.max_size):
12         y = i * box_height
13         if i < len(self.items):
14             color = 'skyblue'
15             label = self.items[i] # 아래에서부터 위로: 맨 먼저 들어온 게 아래쪽
16         else:
17             color = 'white'
18             label = ''
19         ax.add_patch(plt.Rectangle((0, y), 1, box_height, facecolor=color, edgecolor='black'))
20         ax.text(0.5, y + box_height / 2, label, va='center', ha='center', fontsize=10)
21
22     plt.title('Stack View (Top ↑)', fontsize=14)
23     plt.tight_layout()
24     plt.show()
25
26 # 테스트 코드 예시
27 if __name__ == '__main__':
28     stack = Stack()
29
30     # 1~10까지 push
31     for i in range(1, 12):
32         stack.push(f'비료- {i:d}')
33
34     stack.peek()
35     stack.pop()
36     stack.peek()
37
38     print('[empty 확인]:', '비어있음\n' if stack.empty() else '비어있지 않음\n')
39
40     stack.visualize()
```


문제 9. 비료 자동 투입기 - 구현화면

```
swpilotPyhon5 — python src/Process4/4-9/main.py — 100x30
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process4/4-9/main.py
[push] 비료 - 1 추가 됨

[push] 비료 - 2 추가 됨

[push] 비료 - 3 추가 됨

[push] 비료 - 4 추가 됨

[push] 비료 - 5 추가 됨

[push] 비료 - 6 추가 됨

[push] 비료 - 7 추가 됨

[push] 비료 - 8 추가 됨

[push] 비료 - 9 추가 됨

[push] 비료 - 10 추가 됨

[경고] 스택이 가득 찼습니다. 더 이상 push할 수 없습니다.

[peek] 현재 top 항목은 : 비료 - 10

[pop] 비료 - 10 제거 됨

[peek] 현재 top 항목은 : 비료 - 9

[empty 확인]: 비어있지 않음
```

